KIẾN TRÚC PHẦN MỀM (SEI & ISO 25010) — 16:9 WIDESCREEN

QUALITY ATTRIBUTES & ARCHITECTURAL TACTICS

Hệ thống hóa toàn diện các thuộc tính chất lượng phần mềm, kịch bản đặc tả chuẩn SEI 6 thành phần và giải pháp thiết kế kiến trúc

---

Bộ môn & Đào tạo

Khoa CNTT - ĐH Khoa học Tự nhiên (HCMUS)

Giáo trình chuẩn SEI

Software Architecture in Practice (Bass, Clements, Kazman)

Thực chứng hệ thống

Kịch bản kiểm thử & Đánh đổi kiến trúc (Trade-offs)

01 Tổng Quan Quality Attributes & Kịch Bản Chuẩn SEI

NỀN TẢNG KIẾN TRÚC

Bản chất cốt lõi: Tính năng chức năng (*Functionality*) là yêu cầu cơ bản nhưng **không quyết định hình thái kiến trúc**. Nếu chỉ cần chạy được chức năng, một khối mã nguồn duy nhất (Monolith) là đủ. Chính các **Thuộc tính chất lượng (Quality Attributes)** — độ trễ, khả năng chịu tải, an ninh, tính sẵn sàng — mới là động lực thúc đẩy việc phân rã module, lựa chọn style kiến trúc và hạ tầng phân tán.

📐 Vai Trò Trong Đánh Giá Kiến Trúc (ATAM)

- **Định lượng hóa yêu cầu phi chức năng:** Thay thế hoàn toàn các phát biểu cảm tính (như "hệ thống phải nhanh, ổn định") bằng các chỉ số có thể đo lường và kiểm chứng được.
- **Cơ sở lựa chọn Architectural Tactics:** Mỗi thuộc tính chất lượng được đáp ứng thông qua các quyết định thiết kế cụ thể đã được kiểm chứng (Tactics).
- **Nhận diện sự đánh đổi (Trade-offs):** Giúp kiến trúc sư cân bằng giữa các mục tiêu mâu thuẫn ngay từ giai đoạn thiết kế.

🎯 Khung Chuẩn SEI & ISO/IEC 25010

- **Khung chuẩn SEI:** Tiếp cận dựa trên kích thích và phản hồi (Stimulus - Response), gắn liền với phương pháp đánh giá kiến trúc ATAM.
- **Chuẩn ISO/IEC 25010:** Phân loại 8 nhóm đặc tính: Performance, Reliability, Security, Maintainability, Usability, Scalability, Portability, Compatibility.
- **Kịch bản kiến trúc (Scenario):** Cầu nối giao tiếp duy nhất và chuẩn xác giữa Stakeholder, Developer và Architect.

CẤU TRÚC 6 THÀNH PHẦN CHUẨN SEI ĐỂ ĐẶC TẢ MỘT QUALITY ATTRIBUTE SCENARIO

STANDARD SPECIFICATION FRAMEWORK

1. STIMULUS SOURCE
Tác nhân kích thích (Người dùng, Hacker, Lỗi phần cứng, Bộ hẹn giờ).

2. STIMULUS
Sự kiện tác động vào hệ thống (Đột biến tải, Gói tin giả mạo, Crash).

3. ARTIFACT
Thành phần kiến trúc nhận kích thích (Frontend, Gateway, DB).

4. ENVIRONMENT
Trạng thái lúc diễn ra kích thích (Bình thường, Đang quá tải, Deploy).

5. RESPONSE
Hành vi xử lý của kiến trúc (Auto-scale, Cách ly lỗi, Chặn token).

6. RESPONSE MEASURE
Thước đo định lượng (p95 < 1.5s, Tỉ lệ lỗi 0%, Uptime 99.99%).

02 Scalability (Khả Năng Mở Rộng Hệ Thống)

★ TRỌNG TÂM DEMO

Định nghĩa: Khả năng duy trì hoặc nâng cao năng lực phục vụ (thông lượng cao, độ trễ ổn định) khi khối lượng công việc tăng lên, bằng cách **bổ sung thêm tài nguyên tính toán** (CPU, RAM, số lượng instance) mà **không phải tái cấu trúc lại mã nguồn**.

⚙️ Architectural Tactics Đạt Scalability

- **Scale-out (Mở rộng ngang):** Nhân bản container microservice không trạng thái (stateless) phía sau Load Balancer (Nginx, Envoy, Kube-proxy).
- **Decouple State (Tách trạng thái):** Lưu trữ session/cache ra bên ngoài tại Redis Cluster, giúp mọi instance backend đều độc lập.
- **Read-Replication & CQRS:** Phân tách đường đọc và ghi dữ liệu, sử dụng Read-Replicas để chia tải cho cơ sở dữ liệu chính.
- **Asynchronous Decoupling:** Sử dụng Message Queue (Kafka, RabbitMQ) để hấp thụ và san phẳng các đợt tăng tải đột biến.

📊 Chỉ Số Đo Lường & Công Cụ Kiểm Thử

- **Throughput (RPS/TPS):** Số lượng yêu cầu/giao dịch hệ thống xử lý thành công trong 1 giây.
- **Độ trễ phân vị p(95), p(99):** Phản ánh đúng trải nghiệm của nhóm người dùng chậm nhất khi hệ thống đang chịu tải đỉnh.
- **Tỉ lệ lỗi dưới áp lực (Fail Rate):** Tỉ lệ % request bị timeout hoặc trả về HTTP 5xx.
- **Công cụ kiểm thử:** `k6` (Constant Arrival Rate), `Locust`, `Apache JMeter`.

🚀 DỮ LIỆU ĐO ĐẠC THỰC NGHIỆM TRÊN MICROSERVICE FRONTEND (350 req/s cố định trong 20s = 7,000 requests)

Cấu hình thử nghiệm	Tổng Request xử lý	Thành công (HTTP 200)	Thất bại (Timeout > 1.5s)	Độ trễ p(95)	Phân tích bản chất kiến trúc
1 Replica Frontend (0.25 CPU)	5,317 reqs	**228 reqs (4.28%)**	**5,089 reqs (95.71%)**	**> 1.50s (Nghẽn)**	CPU chạm ngưỡng giới hạn, nghẽn hàng đợi TCP, phần lớn request bị drop
3 Replicas Frontend (Nginx Balancer)	6,506 reqs	**6,494 reqs (99.81%)**	**12 reqs (0.18%)**	**1.33s (< 1.5s)**	**Tải phân bổ đều 1/3 cho mỗi pod, giải phóng hoàn toàn hàng đợi, đạt chuẩn SLA**

03 Security (An Ninh & Bảo Mật Hệ Thống)

LÝ THUYẾT VẤN ĐÁP

Định nghĩa: Khả năng của hệ thống tự bảo vệ dữ liệu và tài nguyên chống lại các hành vi truy cập, sửa đổi, phá hoại hoặc đánh cắp trái phép (**Tam giác CIA:** Confidentiality - Tính bí mật, Integrity - Tính toàn vẹn, Availability - Tính sẵn sàng), đồng thời đảm bảo quyền truy cập hợp lệ và khả năng truy vết (Nonrepudiation).

Architectural Tactics Theo Chuẩn SEI

- **Detect Attacks (Phát hiện tấn công):** Giám sát lưu lượng bất thường qua IDS/IPS, WAF; kiểm tra tính toàn vẹn gói tin bằng chữ ký HMAC/Checksum; quét phát hiện Brute-force/DDoS.
- **Resist Attacks (Chống chịu tấn công):** • *Authentication:* Xác thực danh tính mạnh (MFA, JWT mã hóa, OAuth2/OIDC).
  • *Authorization:* Kiểm soát truy cập chặt chẽ (RBAC, ABAC, Principle of Least Privilege).
  • *Encrypt Data:* Mã hóa in-transit (mTLS 1.3) và at-rest (AES-256).
- **React & Recover (Phản ứng & Phục hồi):** Thu hồi token phiên (Session Revocation), cách ly subnet bị xâm nhập, lưu vết Audit Trail phục vụ điều tra.

Nguyên Tắc Kiến Trúc & Phương Pháp Đánh Giá

- **Zero Trust Architecture (ZTA):** Mặc định không tin cậy bất kỳ thành phần nào, kể cả giao tiếp nội bộ giữa các microservices trong mạng LAN.
- **Phòng thủ theo chiều sâu (Defense-in-Depth):** Thiết lập nhiều lớp rào chắn: API Gateway WAF → Service Mesh mTLS → RBAC Service → DB Encryption.
- **Các phương pháp bảo mật mã nguồn & hệ thống:**
  • *SAST (Static Analysis):* Rà soát lỗ hổng trực tiếp trong source code.
  • *SCA (Software Composition Analysis):* Kiểm tra CVE trong các thư viện phụ thuộc.
  • *DAST (Dynamic Analysis):* Kiểm thử hộp đen ứng dụng đang chạy thực tế.

KỊCH BẢN KIẾN TRÚC MẪU (SECURITY SCENARIO): TẤN CÔNG ĐÁNH CẮP PHIÊN & GIẢ MẠO GIAO DỊCH

SEI 6-PART SPECIFICATION

1. STIMULUS SOURCE	2. STIMULUS	3. ARTIFACT	4. ENVIRONMENT	5. RESPONSE	6. RESPONSE MEASURE
Kẻ tấn công từ Internet thông qua kịch bản độc hại XSS/CSRF.	Cố gắng trích xuất Cookie phiên làm việc hoặc giả mạo lệnh gửi giỏ hàng.	API Gateway, Cookie Middleware và Dịch vụ Thanh toán.	Hệ thống đang hoạt động công khai qua môi trường Internet.	Cookie khóa cứng HttpOnly/SameSite; API Gateway từ chối request thiếu CSRF token.	Không lộ thông tin nhạy cảm; hành vi bất thường được ghi log và gửi cảnh báo trong 1s.

04 Availability (Độ Sẵn Sàng & Chống Chịu Sự Cố)

LÝ THUYẾT VẤN ĐÁP

Định nghĩa: Tỉ lệ thời gian hệ thống vận hành đúng chức năng và sẵn sàng đáp ứng yêu cầu người dùng. Trọng tâm cốt lõi là khả năng tự phục hồi khi có sự cố. Cần phân biệt rõ: **Fault** (khiếm khuyết/sự cố cục bộ tại một node) nếu không được cô lập kịp thời sẽ chuyển hóa thành **Failure** (sụp đổ toàn bộ hệ thống).

1. Phát Hiện Lỗi (Detect)

- **Ping / Echo & Heartbeat:** Kiểm tra trạng thái sống còn của node định kỳ qua mạng.
- **Liveness Probe:** Kubernetes tự động kill và restart lại container bị deadlock/treo.
- **Readiness Probe:** Tạm ngừng điều phối lưu lượng đến pod đang khởi động hoặc quá tải.
- **Timeout Detection:** Ngắt kết nối sớm khi dịch vụ phụ thuộc phản hồi quá chậm.

2. Phục Hồi Lỗi (Recover)

- **Active Redundancy (Hot Spare):** Nhiều node chạy song song cùng nhận tải; một node chết, node khác xử lý ngay.
- **Passive Redundancy:** Node dự phòng (Warm/Cold) sẵn sàng khởi động thay thế node chính.
- **Exponential Backoff Retry:** Thử lại kèm giãn cách thời gian tránh làm nghẽn thêm downstream.
- **Graceful Degradation:** Trả về dữ liệu mặc định/fallback khi dịch vụ phụ bị lỗi.

3. Ngăn Ngừa Lỗi (Prevent)

- **Circuit Breaker:** Tự động ngắt luồng gọi tới service đang hỏng, ngăn chặn lỗi domino dây chuyền.
- **ACID Transactions & Sagas:** Bảo toàn trạng thái dữ liệu khi có lỗi xảy ra giữa chừng.
- **Removal from Service:** Tự động cách ly node có dấu hiệu chập chờn trước khi người dùng gặp lỗi.
- **Rate Limiting:** Giữ tải hệ thống luôn nằm dưới ngưỡng chịu đựng an toàn.

KỊCH BẢN KIẾN TRÚC MẪU (AVAILABILITY SCENARIO): XỬ LÝ SỰ CỐ KHI DỊCH VỤ PHỤ BỊ TREO

SEI 6-PART SPECIFICATION

1. STIMULUS SOURCE	2. STIMULUS	3. ARTIFACT	4. ENVIRONMENT	5. RESPONSE	6. RESPONSE MEASURE
Service gợi ý sản phẩm bị crash do lỗi tràn bộ nhớ (OOM).	Hàng nghìn người dùng gửi yêu cầu thanh toán giỏ hàng.	Frontend Gateway và Checkout Microservice.	Hệ thống đang phục vụ bình thường trong giờ cao điểm.	Circuit breaker ngắt gọi service phụ, trả về fallback, tập trung tài nguyên cho thanh toán.	Giao dịch thanh toán thành công 100%, uptime toàn hệ thống duy trì mức 99.99%.

05 Performance (Hiệu Năng & Đáp Ứng Thời Gian)

LÝ THUYẾT VẤN ĐÁP

Định nghĩa: Khả năng của hệ thống đáp ứng các ràng buộc về mặt thời gian (timing requirements) khi tiếp nhận các sự kiện đầu vào. Cần phân biệt rành mạch: **Performance** phản ánh tốc độ/độ trễ tại một mức tải nhất định; còn **Scalability** là năng lực duy trì Performance đó khi tải tăng trưởng liên tục.

Tactic: Kiểm Soát Nhu Cầu (Control Demand)

- **Manage Sampling Rate:** Giảm tần suất đẩy dữ liệu telemetry/giám sát không cần thiết về máy chủ.
- **Event Prioritization:** Sử dụng hàng đợi ưu tiên (Priority Queue) để tác vụ người dùng trực tiếp được xử lý trước các tác vụ nền (batch jobs).
- **Giảm Overhead Giao Thức:** Chuyển đổi từ định dạng JSON/HTTP/1.1 dạng văn bản cồng kềnh sang giao thức nhị phân gRPC/Protobuf trên nền HTTP/2 đa luồng (Multiplexing).
- **Bound Queue Sizes:** Giới hạn độ dài hàng đợi nhằm ngăn chặn cạn kiệt bộ nhớ khi lưu lượng bị tắc nghẽn.

Tactic: Quản Lý Tài Nguyên (Manage Resources)

- **Xử Lý Bất Đồng Bộ Non-blocking:** Sử dụng kiến trúc hướng sự kiện (Event Loop, Goroutines) giải phóng thread thay vì blocking I/O truyền thống.
- **Caching Đa Tầng (Multi-layer Caching):** Cache tại RAM ứng dụng, Cache phân tán Redis và Cache biên CDN cho toàn bộ tài nguyên tĩnh.
- **Data Replication:** Duy trì các bản sao dữ liệu đọc cục bộ gần máy chủ xử lý để giảm thiểu độ trễ truy vấn qua mạng.
- **Connection Pooling:** Duy trì kết nối TCP và Database socket (keep-alive) nhằm triệt tiêu chi phí bắt tay 3 bước lặp đi lặp lại.

Ba Trục Chỉ Số Đo Lường Hiệu Năng Cốt Lõi Khi Trả Lời Vấn Đáp

1. Latency vs. Throughput:
• *Latency (Độ trễ):* Thời gian hoàn thành 1 request (ms).
• *Throughput (Thông lượng):* Số request hoàn tất / giây.

2. Percentiles (Độ trễ phân vị):
• *Average:* Dễ bị sai lệch bởi các giá trị ngoại lai.
• *p(95), p(99):* Chuẩn đánh giá độ trễ của nhóm người dùng chậm nhất.

3. Bão hòa tài nguyên (Saturation):
• Mức độ sử dụng % CPU, RAM, Disk I/O, Network.
• Định luật Amdahl về giới hạn song song hóa.

06 Modifiability (Khả Năng Sửa Đổi & Bảo Trì)

LÝ THUYẾT VẤN ĐÁP

Định nghĩa: Chi phí, thời gian và mức độ rủi ro khi thực hiện các thay đổi đối với hệ thống phần mềm (sửa lỗi, bổ sung tính năng nghiệp vụ mới, thay thế nền tảng công nghệ) mà không gây ảnh hưởng tiêu cực (Side Effects) đến các chức năng đang vận hành bình thường.

🧩 Architectural Tactics Giảm Chi Phí Thay Đổi (SEI)

- **Reduce Size of Module:** Chia nhỏ module theo nguyên lý Đơn trách nhiệm (Single Responsibility Principle - SRP), mỗi module chỉ thực hiện một nhiệm vụ duy nhất.
- **Increase Cohesion:** Đảm bảo toàn bộ mã nguồn bên trong một module có mối liên kết nghiệp vụ chặt chẽ, phục vụ cùng một mục tiêu cụ thể.
- **Reduce Coupling (Giảm phụ thuộc):** • Sử dụng Interface và Dependency Injection để đảo ngược quyền phụ thuộc.
  • Sử dụng thành phần trung gian (Broker, API Gateway, Event Bus) để che giấu kiến trúc.
  • Kiến trúc Publish-Subscribe tách rời hoàn toàn nơi phát và nơi nhận dữ liệu.
- **Defer Binding Time:** Dời thời điểm liên kết sang lúc triển khai: biến môi trường `.env`, Dynamic Plugin Loading, Feature Flags bật/tắt tính năng runtime.

📉 Hiệu Ứng Gợn Sóng (Ripple Effect) & Đo Lường

- **Ripple Effect:** Rủi ro khi sửa đổi một module kéo theo việc phải sửa đổi hàng loạt module khác liên quan. Một kiến trúc chuẩn mực phải triệt tiêu hiệu ứng gợn sóng ngay tại ranh giới module.
- **Các chỉ số định lượng đánh giá Modifiability:**
  • *Number of affected modules:* Số lượng module bị ảnh hưởng khi có một yêu cầu thay đổi.
  • *Cyclomatic Complexity (CC):* Độ phức tạp luồng điều khiển của thuật toán.
  • *Coupling Between Objects (CBO):* Mức độ gắn kết giữa các lớp đối tượng.
  • *Person-hours per change:* Số giờ công thực tế để hoàn thành và kiểm thử thay đổi.

KỊCH BẢN KIẾN TRÚC MẪU (MODIFIABILITY SCENARIO): TÍCH HỢP CỔNG THANH TOÁN TRỰC TUYẾN MỚI

SEI 6-PART SPECIFICATION

1. STIMULUS SOURCE	2. STIMULUS	3. ARTIFACT	4. ENVIRONMENT	5. RESPONSE	6. RESPONSE MEASURE
Đội ngũ phát triển phần mềm / Yêu cầu kinh doanh mới.	Bổ sung phương thức thanh toán ví điện tử mới (Momo/VNPay).	Payment Service và Checkout Service.	Giai đoạn phát triển mã nguồn (Design & Build time).	Chỉ cần cài đặt Interface `IPaymentProvider` mới, cấu hình qua file `.env`.	Không chạm vào code của Checkout service, hoàn thành tích hợp và test dưới 2 ngày công.

07 Testability (Khả Năng Kiểm Thử Kiến Trúc)

LÝ THUYẾT VẤN ĐÁP

Định nghĩa: Mức độ dễ dàng mà phần mềm cho phép người phát triển/kiểm thử thiết kế các kịch bản thực thi nhằm phát hiện lỗi. Hệ thống có Testability cao khi nó **"dễ dàng bộc lộ khuyết tật" (gives up its faults easily)** thông qua quy trình kiểm thử tự động.

🧪 Architectural Tactics Nâng Cao Testability (SEI)

- **Control & Observe State (Kiểm soát & Quan sát trạng thái):** • Cung cấp giao diện kiểm thử chuyên biệt (Specialized Test Interfaces, Debug endpoints).
  • Xây dựng endpoint hỗ trợ reset trạng thái cơ sở dữ liệu về Fixture mẫu ban đầu.
  • Cung cấp Health Check API chi tiết tình trạng kết nối tới các dịch vụ phụ thuộc.
- **Abstract Data Sources (Trừu tượng hóa tầng dữ liệu):** • Sử dụng Repository Pattern tách rời tầng truy cập dữ liệu, cho phép thay thế bằng Mock/Stub trong Unit Test mà không cần kết nối tới Database vật lý.
- **Limit Structural Complexity (Kiểm soát độ phức tạp):** • Triệt tiêu triệt để các phụ thuộc vòng (Circular dependencies).
  • Hạn chế tính bất định (Non-determinism) trong xử lý đa luồng đồng thời.

📈 Kim Tự Tháp Kiểm Thử & Chỉ Số Đo Lường

- **Kim tự tháp kiểm thử chuẩn (Test Pyramid):** • *Unit Tests (70%):* Nhanh, chi phí thấp, kiểm thử cô lập từng hàm logic.
  • *Integration Tests (20%):* Kiểm thử giao tiếp hợp đồng giữa các microservices.
  • *End-to-End Tests (10%):* Chậm, tốn kém, mô phỏng toàn bộ luồng người dùng thực tế.
- **Chỉ số đo lường Testability:**
  • *Code Coverage:* Tỉ lệ % dòng lệnh (Statement) và rẽ nhánh (Branch) được test bao phủ.
  • *Test Suite Execution Time:* Thời gian chạy bộ test trên CI/CD (phải dưới 10 phút).
  • *Fault Isolation:* Tốc độ định vị nguyên nhân gốc khi một test case bị thất bại.

KỊCH BẢN KIẾN TRÚC MẪU (TESTABILITY SCENARIO): KIỂM THỬ TỰ ĐỘNG CART SERVICE BẰNG MOCK STORE

SEI 6-PART SPECIFICATION

1. STIMULUS SOURCE	2. STIMULUS	3. ARTIFACT	4. ENVIRONMENT	5. RESPONSE	6. RESPONSE MEASURE
Hệ thống CI/CD pipeline khi có Pull Request được tạo mới.	Kích hoạt toàn bộ bộ kiểm thử tự động Unit & Integration Test.	Cart Service và Mock Interface `ICartStore`.	Môi trường container CI cô lập (không có Redis cluster thật).	Mock store giả lập thành công mọi tình huống lỗi kết nối và timeout.	Branch coverage đạt > 85%, toàn bộ test suite hoàn tất trong dưới 60 giây.

08 Deployability & Usability

LÝ THUYẾT VẤN ĐÁP

🚀 1. Deployability (Khả Năng Triển Khai)

Khả năng đưa bản phát hành phần mềm mới vào môi trường vận hành thực tế trong thời gian dự đoán được, với công sức tối thiểu và không gây gián đoạn người dùng.

- **Tactics Triển Khai Không Downtime (Zero-Downtime):**
  • *Containerization:* Đóng gói đồng nhất qua Docker, triệt tiêu lỗi không tương thích môi trường.
  • *Blue-Green Deployment:* Duy trì 2 môi trường song song; chuyển đổi router lưu lượng tức thời.
  • *Canary Release:* Phát hành bản mới cho 5% người dùng trước khi triển khai 100%.
  • *Automated Rollback:* Tự động quay về phiên bản ổn định cũ nếu tỉ lệ lỗi HTTP 5xx tăng vọt.
- **Chỉ số DORA:** Deployment Frequency (tần suất deploy), Lead Time for Changes (thời gian đưa code lên production), Change Failure Rate.

👤 2. Usability (Khả Năng Sử Dụng)

Mức độ dễ dàng, trực quan và thuận tiện để người dùng hoàn thành mục tiêu công việc, cùng các cơ chế trợ giúp tương tác từ hệ thống.

- **Tactics Usability Cốt Lõi Theo Giáo Trình SEI:**
  • *Maintain Task Model:* Hệ thống hiểu ngữ cảnh tác vụ để chủ động hỗ trợ (gợi ý tìm kiếm Type-ahead, tự động điền form, tự động sửa lỗi chính tả).
  • *Maintain User Model:* Ghi nhớ thói quen, quyền hạn và tùy biến cá nhân hóa trải nghiệm theo từng phân nhóm người dùng.
  • *Maintain System Model:* Phản hồi trạng thái minh bạch cho người dùng (Thanh tiến trình, thông báo lỗi thân thiện, hỗ trợ Undo/Redo tác vụ).
- **Chỉ số đo lường:** Task completion time, Error recovery rate, Điểm chuẩn hóa khảo sát SUS (System Usability Scale).

💡 Tác Động Trực Tiếp Của Kiến Trúc Đến Usability (Không Đơn Thuần Là UI/UX)

Usability không chỉ dừng lại ở giao diện đồ họa bên ngoài mà chịu sự chi phối chặt chẽ từ các quyết định kiến trúc: Tính năng **Hủy tác vụ (Cancel)** đòi hỏi kiến trúc phải hỗ trợ ngắt luồng I/O đang xử lý; tính năng **Hoàn tác (Undo)** đòi hỏi kiến trúc phải lưu trữ trạng thái qua Command Pattern hoặc Memento Pattern; và tính năng phản hồi trạng thái thời gian thực đòi hỏi hạ tầng kết nối **Websocket / SSE** liên tục từ máy chủ backend.

09 Ma Trận Đánh Đổi Kiến Trúc (Architectural Trade-offs)

TỔNG HỢP & VẤN ĐÁP

Chân lý kiến trúc phần mềm: Không tồn tại một kiến trúc "hoàn hảo" cho mọi thuộc tính chất lượng. Việc tối ưu hóa một thuộc tính chất lượng hầu như luôn dẫn đến sự suy giảm của một hoặc nhiều thuộc tính chất lượng khác.

Cặp Đánh Đổi Cốt Lõi	Bản Chất Mâu Thuẫn Kiến Trúc	Chiến Lược Cân Bằng Thực Tế (Tactics)
Scalability vs. Consistency (Định lý CAP)	Khi scale-out dữ liệu sang hàng chục node phân tán để chịu tải cao, việc bắt buộc tất cả node phải đồng bộ dữ liệu tức thời (Strong Consistency) sẽ làm nghẽn mạng và sụt giảm nghiêm trọng Availability.	Chấp nhận mô hình **Eventual Consistency** (Nhất quán cuối cùng). Áp dụng *Saga Pattern* hoặc Outbox Pattern với Message Queue thay cho Distributed Two-Phase Commit (2PC).
Security vs. Performance	Việc áp dụng mã hóa toàn diện (mTLS), phân tích chữ ký số, giải mã payload và kiểm tra token trên từng request sẽ tiêu tốn chu kỳ CPU và tăng độ trễ mạng (Network Latency).	Dùng Hardware Crypto Acceleration (TLS offloading tại API Gateway), Cache token xác thực phía Gateway, chỉ áp dụng mTLS nội bộ cho luồng dữ liệu tài chính cực kỳ nhạy cảm.
Performance vs. Modifiability	Tối ưu hiệu năng cực đoan bằng cách gộp chung module, viết code bậc thấp, chia sẻ trực tiếp bộ nhớ hoặc nhúng cache đa tầng khiến code phức tạp, gắn kết chặt chẽ và rất khó sửa đổi.	Áp dụng Clean Architecture / Microservices phân tách ranh giới rõ ràng. Chỉ tối ưu cục bộ tại đúng **điểm nghẽn cổ chai (Bottleneck)** sau khi có kết quả Profiling thực tế.
Availability vs. Financial Cost	Để đạt mức sẵn sàng "năm số 9" (99.999% - chỉ được phép downtime 5 phút/năm), hệ thống bắt buộc phải nhân bản toàn bộ hạ tầng Active-Active trên nhiều Data Center khác nhau, chi phí đội lên gấp nhiều lần.	Xác định rõ SLA/SLO theo giá trị nghiệp vụ: Chấp nhận 99.9% ("ba số 9") cho các dịch vụ phụ (gợi ý sản phẩm, giỏ hàng), chỉ đầu tư 99.99% cho luồng thanh toán cốt lõi.

🎓 **THÔNG ĐIỆP KẾT LUẬN VẤN ĐÁP:**

"Mục tiêu của người kiến trúc sư phần mềm không phải là xây dựng một hệ thống tối đa hóa tất cả các chỉ số chất lượng, mà là **nhận diện chính xác mục tiêu nghiệp vụ tối thượng của hệ thống**, từ đó đưa ra các quyết định đánh đổi kiến trúc (Trade-offs) có cơ sở khoa học và định lượng được bằng kịch bản kiểm thử."